

Lecture 8 -- Logical Agents

- 基于知识的智能体（Knowledge-based agents）
 - 一个基于知识的智能体由两部分组成：
 - 知识库（Knowledge base）：领域特定的内容（domain-specific）。
 - 推理机制（Inference mechanism）：领域无关的算法（domain-independent）。
 - 智能体必须能够：
 - 表示状态、动作等。
 - 融入新的感知（新观测）。
 - 更新内部的世界表示。
 - 推断世界的隐藏属性。
 - 推断适当的动作。
 - 构建智能体的声明式（declarative）方法：
 - 添加新句子：Tell（告诉）知识库它需要知道的东西。
 - 查询已知信息：Ask（询问）自身该做什么——答案应当从知识库推出（follow from the KB）。

Knowledge-based agents

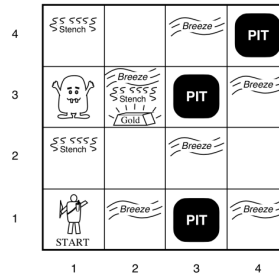
输入：当前时刻智能体接收到的感知

```
function KB-AGENT(percept) returns an action
  static: KB, a knowledge base
           t, a counter, initially 0, indicating time
  TELL(KB, MAKE-PERCEPT-SENTENCE(percept, t))
  action ← ASK(KB, MAKE-ACTION-QUERY(t))
  TELL(KB, MAKE-ACTION-SENTENCE(action, t))
  t ← t + 1
  return action
```

- 将当前感知转换为适合存入知识库的句子（Make-Percept-Sentence），并用 TELL 操作把它加入到 KB。
- 将当前情形（可能包括时间 *t*）构造为一个动作查询（Make-Action-Query），然后用 ASK 向 KB 查询/推理，得到应该采取哪个动作。
- 把选择的动作也记录到知识库中（用句子形式），例如记录“在时间 *t* 执行了 *action*”或记录动作预期的效果。

The Wumpus World

- 4 X 4 grid of rooms
- Squares adjacent to Wumpus are smelly and squares adjacent to pit are breezy
- Glitter iff gold is in the same square
- Shooting kills Wumpus if you are facing it
- Wumpus emits a horrible scream when it is killed that can be heard anywhere
- Shooting uses up the only arrow
- Grabbing picks up gold if in same square
- Releasing drops the gold in same square



- 性能度量（Performance measure）：获得黄金 +1000；死亡（被吃或掉入坑） -1000；每执行一个动作 -1；使用箭 -10。游戏在智能体死亡或走出洞穴时结束。
- 环境（Environment）：4×4 的房间格子；智能体从格子 [1,1] 出发，面向向右；黄金和 Wumpus 的位置从除 [1,1] 以外的所有格子中均匀随机选择；除起始格外的每个格子以 0.2 的概率为坑。
- 执行器（Actuators）：左转、右转、前进、抓取、释放、射击。
- 传感器（Sensors）：臭气（Stench）、微风（Breeze）、闪光（Glitter）、撞击（Bump）、尖叫（Scream）。
 - 传感器信息表示为一个 5 元素列表。
 - 例如：[Stench, Breeze, None, None, None] → [臭气, 微风, 无, 无, 无]。
- Wumpus World 的属性：部分可观测（Partially observable）静态（Static）离散（Discrete）单智能体（Single-agent）确定性（Deterministic）顺序/序列式（Sequential）

Logic

- 知识库（Knowledge base）：以形式表示（逻辑）的一组句子。
- 逻辑（Logics）：用于表示知识并推导结论的形式语言。
 - 句法（Syntax）：定义语言中格式正确的句子。
 - 语义（Semantic）：定义句子在某个世界中的真值或含义。
- 推理（Inference）：从已有句子推导出新句子的过程。
- 逻辑蕴含（Logical entailment）：句子之间的一种关系。
 - 某个句子从其他句子“逻辑上推出”。记作 $KB \models \alpha$ （即：从知识库 KB 可以逻辑推出 α ）。
- 命题逻辑（Propositional logic, PL）
 - 句法（Syntax）：定义允许的句子或命题的形式。
 - 定义（命题 Proposition）：命题是一个陈述句，具有真或假的值。
 - 语义（semantics）：定义了判断句子真假的规则。
 - 真值表 Truth tables
 - 语义可以用真值表来指定。
 - 布尔值域：T（真）、F（假）

- 真值表的行数： $R = 2^n$ 。
- 原子命题（Atomic proposition）： 单个命题符号。每个符号表示一个命题。表示法通常为写字母，并可带下标。
 - 原子命题的例子：
 - “ $2+2=4$ ” 是一个真命题。
 - $W_{1,3}$ 是一个命题 —— 当且仅当位置 [1,3] 有 Wumpus 时该命题为真。
 - “你好吗？”或“你好！”不是命题（因为它们不是可判定为真或假的陈述）。一般来说，问题、命令或主观意见不是命题。
- 复合命题（Compound proposition）： 由原子命题通过括号与逻辑连接词构造而成。
 - 设 p 、 p_1 和 p_2 为命题。
 - 否定（Negation）： $\neg p$ 也是一个命题。我们把原子命题或它的否定称为“文字”（literal）。例如， $W_{1,3}$ 是一个正文字， $\neg W_{1,3}$ 是一个负文字。
 - - **Conjunction** $p_1 \wedge p_2$. E.g., $W_{1,3} \wedge P_{3,1}$
 - **Disjunction** $p_1 \vee p_2$. E.g., $W_{1,3} \vee P_{3,1}$
 - **Implication** $p_1 \rightarrow p_2$. E.g., $W_{1,3} \wedge P_{3,1} \rightarrow \neg W_{2,2}$
 - **If and only if** $p_1 \leftrightarrow p_2$. E.g., $W_{1,3} \leftrightarrow \neg W_{2,2}$

Building propositions

Negation:

p	$\neg p$
T	F
F	T

Conjunction:

p	q	$p \wedge q$
T	T	T
T	F	F
F	T	F
F	F	F

Disjunction:

p	q	$p \vee q$
T	T	T
T	F	T
F	T	T
F	F	F

Exclusive or: $p \oplus q \equiv (p \wedge \neg q) \vee (\neg p \wedge q)$

p	q	$p \oplus q$
T	T	F
T	F	T
F	T	T
F	F	F

Implication:

p	q	$p \rightarrow q$
T	T	T
T	F	F
F	T	T
F	F	T

Biconditional or If and only if (IFF): $p \leftrightarrow q$ 同为真/同为假

p	q	$p \leftrightarrow q$
T	T	T
T	F	F
F	T	F
F	F	T

p	q	r	$\neg r$	$p \vee q$	$p \vee q \rightarrow \neg r$
T	T	T	F	T	F
T	T	F	T	T	T
T	F	T	F	T	F
T	F	F	T	T	T
F	T	T	F	T	F
F	T	F	T	T	T
F	F	T	F	F	T
F	F	F	T	F	T

运算符优先级:

- 与算术运算符类似，逻辑运算符在求值时也有优先级，顺序如下：
 - 括号内表达式（从内到外）
 - 否定 (Negation)
 - 与 (AND)
 - 或 (OR)
 - 蕴含 (Implication)
 - 双向蕴含 / 等价 (Biconditional)
 - 从左到右 (Left to right, 若优先级相同则按左到右)
- 如果有任何疑问，请使用括号来明确运算顺序！

- 逻辑等价 Logical equivalence:

- 如果两个命题 p 与 q 在真值表中对应列的真值完全相同，则称它们逻辑等价 (logically equivalent)。
- 我们记作 $p \equiv q$ 。
- $\neg p \vee q \equiv p \rightarrow q$

p	q	$\neg p$	$\neg p \vee q$	$p \rightarrow q$
T	T	F	T	T
T	F	F	F	F
F	T	T	T	T
F	F	T	T	T

Properties of operators

- Commutativity: 交换律
 - $p \wedge q \equiv q \wedge p$
 - $p \vee q \equiv q \vee p$
- Associativity: 结合律
 - $(p \wedge q) \wedge r \equiv p \wedge (q \wedge r)$
 - $(p \vee q) \vee r \equiv p \vee (q \vee r)$
- Identity element:
 - $p \wedge \text{True} \equiv p$
 - $p \vee \text{True} \equiv \text{True}$
- $\neg(\neg p) \equiv p$
- $p \wedge p \equiv p$ and $p \vee p \equiv p$
- Distributivity: 分配律
 - $p \wedge (q \vee r) \equiv (p \wedge q) \vee (p \wedge r)$
 - $p \vee (q \wedge r) \equiv (p \vee q) \wedge (p \vee r)$
- $p \wedge (\neg p) \equiv \text{False}$ and $p \vee (\neg p) \equiv \text{True}$
- DeMorgan's laws:
 - $\neg(p \wedge q) \equiv (\neg p) \vee (\neg q)$
 - $\neg(p \vee q) \equiv (\neg p) \wedge (\neg q)$

P	$\neg p$	$p \vee \neg p$	$p \wedge \neg p$
T	F	T	F
F	T	T	F

- 永真式 (Tautology): 始终为真
- 矛盾式 (Contradiction): 始终为假
- 或然式 (Contingency): 有时真有时假

- 给定一个 implication: $p \rightarrow q$

- Converse (逆命题): $q \rightarrow p$
- Contrapositive (逆否命题): $\neg q \rightarrow \neg p$
- Inverse (否命题/逆命题的否定): $\neg p \rightarrow \neg q$

Inference (Modus Ponens)

$$\frac{p \quad p \rightarrow q}{q}$$

Horn clauses: a proposition of the form:

$$p_1 \wedge \dots \wedge p_n \rightarrow q$$

Modus Ponens deals with **Horn clauses**:

$$\frac{p_1, \dots, p_n \quad (p_1 \wedge \dots \wedge p_n) \rightarrow q}{q}$$

$$\frac{\neg q \quad p \rightarrow q}{\neg p}$$

$$\frac{\neg beach \quad hot \rightarrow beach}{\neg hot}$$

Common Rules

- Addition: $\frac{p}{p \vee q}$
- Simplification: $\frac{p \wedge q}{q}$
- Disjunctive-syllogism: $\frac{p \vee q \quad \neg p}{q}$
- Hypothetical-syllogism: $\frac{p \rightarrow q \quad q \rightarrow r}{p \rightarrow r}$

Wumpus world KB

1,4	2,4	3,4	4,4
1,3	2,3	3,3	4,3
1,2	2,2 P?	3,2	4,2
OK			
1,1 V OK	2,1 B OK	3,1 P?	4,1

- Let's build the KB for the reduced Wumpus world.

- Let P_{ij} be true if there is a pit in $[i, j]$

- Let B_{ij} be true if there is a breeze in $[i, j]$

$$R_1: \neg P_{1,1}$$

- "A square is breezy if and only if there is an adjacent pit"

$$R_2: B_{1,1} \Leftrightarrow P_{1,2} \vee P_{2,1}$$

$$R_3: B_{2,1} \Leftrightarrow P_{1,1} \vee P_{2,2} \vee P_{3,1}$$

$$R_4: \neg B_{1,1}$$

$$R_5: B_{2,1}$$

- 语义 (Semantics) : 通过模型检测 (Model Checking) 来判定蕴含关系

- 枚举所有模型并证明目标句子在所有模型中都成立。

- 记作 $KB \models \alpha$

- 句法 (Syntax) : 通过定理证明 (Theorem Proving) 来判定蕴含关系

- 对知识库应用推理规则, 构造 α 的证明, 而不必枚举并检查所有模型。

- 记作 $KB \vdash \alpha$

- Soundness & Completeness / 正确性与完备性

- Sound (正确) : 不会推导出假的公式, 即只推导出被知识库蕴含的句子。

$$\{\alpha \mid KB \vdash \alpha\} \subseteq \{\alpha \mid KB \models \alpha\}$$

- Complete (完备) : 能推导出所有被知识库蕴含的句子。

$$\{\alpha \mid KB \vdash \alpha\} \supseteq \{\alpha \mid KB \models \alpha\}$$

- Validity & satisfiability / 有效性与可满足性

- valid** 当且仅当它在所有模型中都为真, 例如 True、 $p \vee \neg p$ 、 $p \Rightarrow p$ 、 $(p \wedge (p \Rightarrow q)) \Rightarrow q$ 。

- 有效性与推理通过“演绎定理” (Deduction Theorem) 相关联: $KB \vdash \alpha$ 当且仅当 $(KB \Rightarrow \alpha)$ 是有效的。

- satisfiable** 当且仅当存在某个模型使其为真, 例如 $p \vee p, r$ 。

- unsatisfiable** 当且仅当在任何模型中都不为真, 例如 $p \wedge \neg p$ 。

- 可满足性与推理的关系: $KB \models \alpha$ IFF $(KB \wedge \neg \alpha)$ 不可满足

•

证明 ($\Rightarrow$): 若 $KB \models \alpha$, 则 $(KB \wedge \neg \alpha)$ 不可满足

1. 假设 $KB \models \alpha$ 。

2. 反设 $(KB \wedge \neg \alpha)$ 可满足, 则存在某模型 M 使 $M \models (KB \wedge \neg \alpha)$ 。

3. 由 $M \models (KB \wedge \neg \alpha)$ 可得 $M \models KB$ 且 $M \models \neg \alpha$ 。

4. 由 $KB \models \alpha$, 任何满足 KB 的模型都必须满足 α , 因此应有 $M \models \alpha$ 。

5. 但第 3 步又给出 $M \models \neg \alpha$, 矛盾 (α 与 $\neg \alpha$ 不能同时为真)。

6. 因此假设错误, $(KB \wedge \neg \alpha)$ 必不可满足。QED.

•

证明 ($\Leftarrow$): 若 $(KB \wedge \neg\alpha)$ 不可满足, 则 $KB \models \alpha$

1. 假设 $(KB \wedge \neg\alpha)$ 不可满足, 即不存在模型 M 使 $M \models KB \wedge \neg\alpha$ 。
2. 反设 $KB \not\models \alpha$ (即 KB 不蕴含 α), 则存在某模型 M_0 使 $M_0 \models KB$ 且 $M_0 \not\models \alpha$ 。
3. 从 $M_0 \not\models \alpha$ 可等价得到 $M_0 \models \neg\alpha$, 于是 M_0 同时满足 KB 与 $\neg\alpha$, 故 $M_0 \models (KB \wedge \neg\alpha)$ 。
4. 这和第 1 步 (无任何满足 $(KB \wedge \neg\alpha)$ 的模型) 矛盾。
5. 因此假设错误, 必须有 $KB \models \alpha$ 。QED.

- 判定蕴含 (Determining entailment)

- 给定一个知识库 (KB) (一组命题逻辑的句子) 和一个查询 α , 输出 KB 是否蕴含 α , 记作: $KB \models \alpha$ 。
- 我们将看到在命题逻辑中做证明的两种方法:
 - 模型检验 (Model checking): 枚举模型 (真值表枚举, 代价呈指数增长)。
 - 用真值表做推理。
 - 模型 = 对每个命题符号赋真/假。
 - 检查: 在所有满足 KB 的模型里, 目标句子是否都为真。
 - 应用推理规则 (Application of inference rules, 或定理证明 / proof checking): 用类似 Modus Ponens 的语法推理规则做导出 (可做自顶向下或自底向上, 前向/后向链推)。证明就是一系列推理规则的应用序列。
 - 把推理看成搜索问题 (Inference as a search problem)
 - 初始状态 (Initial state): 初始的知识库 KB。
 - 动作 (Actions): 对 KB 中匹配到推理规则前半部分的所有句子, 应用这些推理规则 (即把规则顶端匹配到的前提作为触发条件)。
 - 结果 (Results): 把推理规则下半部分 (结论) 对应的句子加入到状态 (即加入 KB)。
 - 目标 (Goal): 到达包含我们想证明的句子的某个状态 (证明完成)。
- 定理证明 (Theorem proving):
 - 推理是正确的 (sound)
 - 在许多情况下, 搜索证明比枚举模型更高效 (因为可以忽略与目标无关的信息)。
 - 真值表有指数级的模型数量。
 - 推理的思想是反复把推理规则应用到 KB 上。
 - 当在 KB 中找到合适的前提时就可以应用推理 (可随时使用)。
 - 两种保证完备性的方法:
 - 归结证明 (Proof by resolution): 使用强有力的推理规则 (归结规则)。
 - 正向或反向链式推理 (Forward or Backward chaining): 在受限形式的命题 (Horn 子句) 上使用肯定前件法 (modus ponens)。
 - 归结 (Resolution): 只有一个单一的推理规则。
 - 归结 + 搜索 = 完备的推理算法。

Proof by Resolution

- **Unit resolution:**

$$\frac{\ell_1 \vee \dots \vee \ell_k \quad m}{\ell_1 \vee \dots \vee \ell_{i-1} \vee \ell_{i+1} \vee \dots \vee \ell_k}$$

where ℓ_i and m are complementary literals. ℓ_i 与 m 互补

- Example:

$$\frac{P_{1,3} \vee P_{2,2} \quad \neg P_{2,2}}{P_{1,3}}$$

- We call a **clause** a disjunction of literals.
- Unit resolution: Clause + Literal = New clause.

- **Resolution inference rule (for CNF):**

$$\frac{\ell_1 \vee \dots \vee \ell_k \quad m_1 \vee \dots \vee m_n}{\ell_1 \vee \dots \vee \ell_{i-1} \vee \ell_{i+1} \vee \dots \vee \ell_k \vee m_1 \vee \dots \vee m_{j-1} \vee m_{j+1} \vee \dots \vee m_n}$$

where ℓ_i and m_j are complementary literals.

- Resolution **applies only to clauses**

- Fact: 命题逻辑 (PL) 中的每个句子在逻辑上都等价于若干子句的 conjunction (即可以转换为子句的合取)。
- 合取范式 (Conjunctive Normal Form, CNF) : 是若干文字 disjunction 的 conjunction。
例子: $(A \vee \neg B) \wedge (B \vee \neg C \vee \neg D)$ 。
- 对于命题逻辑, 针对 CNF 的归结推理规则是正确且完备的。

Conversion to CNF

$$B_{1,1} \Leftrightarrow (P_{1,2} \vee P_{2,1})$$

1. Eliminate $\Leftrightarrow$, replacing $\alpha \Leftrightarrow \beta$ with $(\alpha \Rightarrow \beta) \wedge (\beta \Rightarrow \alpha)$.
 $(B_{1,1} \Rightarrow (P_{1,2} \vee P_{2,1})) \wedge ((P_{1,2} \vee P_{2,1}) \Rightarrow B_{1,1})$
2. Eliminate $\Rightarrow$, replacing $\alpha \Rightarrow \beta$ with $\neg \alpha \vee \beta$.
 $(\neg B_{1,1} \vee P_{1,2} \vee P_{2,1}) \wedge (\neg (P_{1,2} \vee P_{2,1}) \vee B_{1,1})$
3. Move $\neg$ inwards using DeMorgan's rules and double-negation:
 $(\neg B_{1,1} \vee P_{1,2} \vee P_{2,1}) \wedge ((\neg P_{1,2} \wedge \neg P_{2,1}) \vee B_{1,1})$
4. Apply distributivity law ($\vee$ over $\wedge$) and flatten:
 $(\neg B_{1,1} \vee P_{1,2} \vee P_{2,1}) \wedge (\neg P_{1,2} \vee B_{1,1}) \wedge (\neg P_{2,1} \vee B_{1,1})$

Resolution algorithm

```
function PL-RESOLUTION( $KB, \alpha$ ) returns true or false
inputs:  $KB$ , the knowledge base, a sentence in propositional logic
         $\alpha$ , the query, a sentence in propositional logic

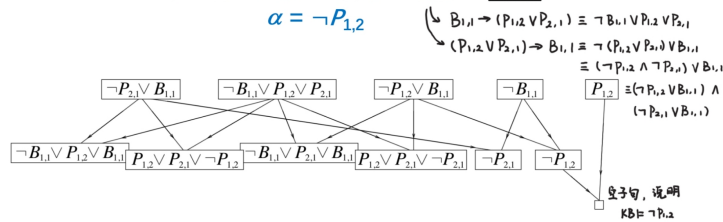
 $clauses \leftarrow$  the set of clauses in the CNF representation of  $KB \wedge \neg \alpha$ 
 $new \leftarrow \{ \}$ 
loop do
  for each  $C_i, C_j$  in  $clauses$  do
     $resolvents \leftarrow$  PL-RESOLVE( $C_i, C_j$ )
    if  $resolvents$  contains the empty clause then return true
     $new \leftarrow new \cup resolvents$ 
  if  $new \subseteq clauses$  then return false
   $clauses \leftarrow clauses \cup new$ 
```

Resolution example

要证明 $KB \models \alpha$, (即 $KB \models \alpha$), 检查 $KB \wedge \neg \alpha$ 是否可满足

$$KB = R_1 \wedge R_2 = (B_{1,1} \leftrightarrow (P_{1,2} \vee P_{2,1})) \wedge \neg B_{1,1}$$

$$\alpha = \neg P_{1,2}$$



正向 / 反向 链式推理 Forward/backward chaining

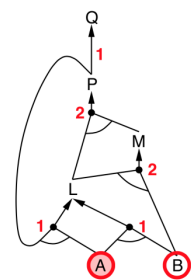
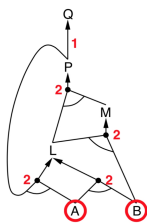
- 知识库由若干 Horn 子句组成
- Horn 子句：形如 $p_1 \wedge \dots \wedge p_n \rightarrow q$ 的命题（前件是若干原子的与，结论是单个原子）
- 肯定前件法（Modus Ponens，针对 Horn 形式）：对于 Horn 知识库是完备的

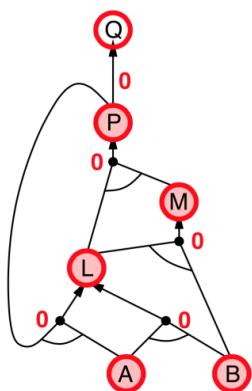
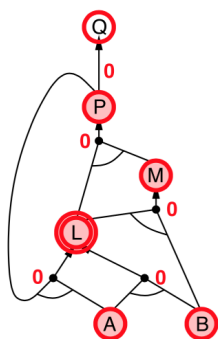
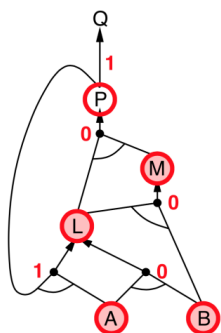
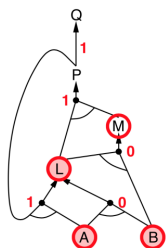
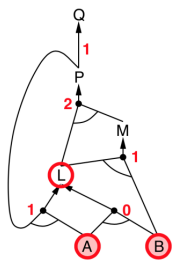
$$\frac{p_1, \dots, p_n \quad (p_1 \wedge \dots \wedge p_n) \rightarrow q}{q}$$

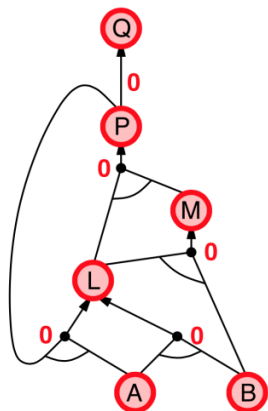
- 正向与反向链在 Horn 子句下为线性时间并且完备。对命题逻辑，归结（resolution）是完备的。
- 正向链（forward chaining）
 - 思路：触发任何其前提在知识库中已被满足的规则，将该规则的结论加入知识库，重复此过程直到找到查询目标。

Forward chaining example

$P \Rightarrow Q$
 $L \wedge M \Rightarrow P$
 $B \wedge L \Rightarrow M$
 $A \wedge P \Rightarrow L$
 $A \wedge B \Rightarrow L$
 A
 B







- 反向链 (backward chaining)

- 思路：从查询 q 向后推演

- 要用反向链证明 q：

- 检查 q 是否已知，或者

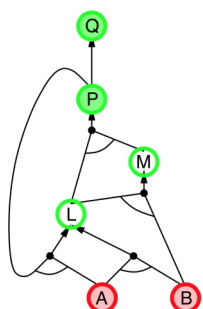
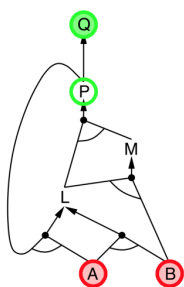
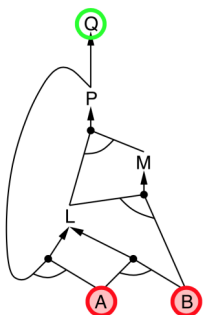
- 选择某条以 q 为结论的规则，反向链证明该规则的所有前提

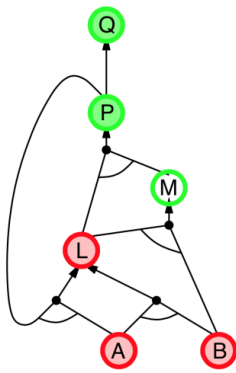
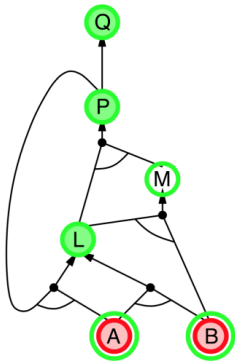
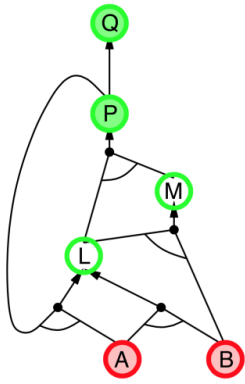
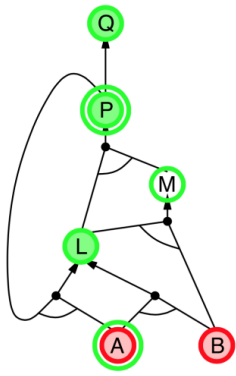
- 避免循环：检查新的子目标是否已在目标栈上

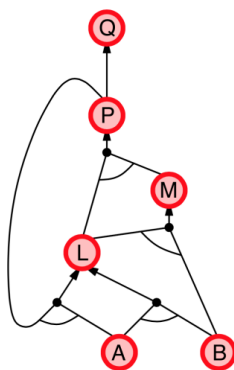
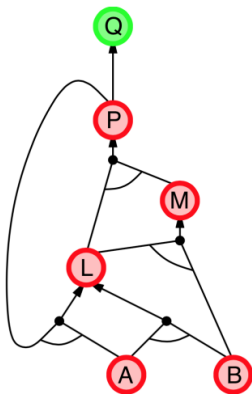
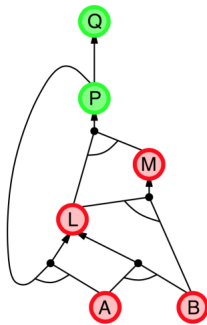
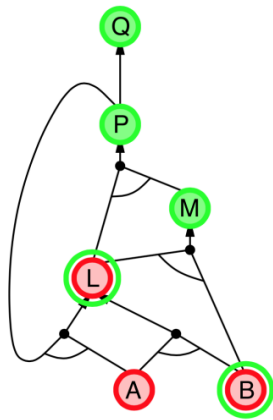
- 避免重复工作：检查新的子目标是否

- 已被证明为真，或

- 已经证明失败







- 正向与反向的比较：
 - 正向链：
 - 数据驱动、自动的、无意识式处理
 - 可能做大量与目标无关的工作
 - 反向链：
 - 目标驱动，适合问题求解

- 反向链的复杂度在很多情况下远小于相对于知识库大小的线性复杂度
- 一阶逻辑 (First Order Logic, FOL)
 - 句法:
 - 项 (terms) 可以是:
 - 常量符号 (例如 A、10、Shenzhen)
 - 变量 (例如 x、y)
 - 由项组成的函数, 例如 $\text{sqrt}(x)$ 、 $\text{sum}(1,2)$
 - 原子公式: 谓词作用于项, 例如 $\text{brother}(x,y)$ 、 $\text{above}(A,B)$
 - 连接词: $\wedge$ 、 $\vee$ 、 $\Rightarrow$ 、 $\Leftrightarrow$ 、 $\neg$
 - 等号: $=$
 - 量词: $\forall$ 、 $\exists$
 - 连接词、等号、量词可以作用于原子公式以构成一阶逻辑的句子。
 - 表示“所有格子都干净”: $\forall x \text{ Square}(x) \Rightarrow \text{Clean}(x)$
 - 表示“存在一些脏的格子”: $\exists x \text{ Square}(x) \wedge \neg \text{Clean}(x)$
 - 备注:
 - $\forall x P(x)$ 类似于 $P(A) \wedge P(B) \wedge \dots$
 - $\exists x P(x)$ 类似于 $P(A) \vee P(B) \vee \dots$
 - $\neg \forall x P(x)$ 等价于 $\exists x \neg P(x)$
 - $\forall x \exists y \text{ likes}(x,y)$ 与 $\exists y \forall x \text{ likes}(x,y)$ 并不等价, 二者含义不同。

First Order Logic

- All birds fly:

$$\forall x \text{ bird}(x) \Rightarrow \text{Fly}(x)$$
- All birds except penguins fly:

$$\forall x \text{ bird}(x) \wedge \neg \text{penguin}(x) \Rightarrow \text{Fly}(x)$$
- Every kid likes candy:

$$\forall x \text{ Kid}(x) \Rightarrow \text{Likes}(x, \text{candy})$$
- Some kids like candy:

$$\exists x \text{ Kid}(x) \wedge \text{Likes}(x, \text{candy})$$
- Brothers are sibling:

$$\forall x, y \text{ Brothers}(x, y) \Rightarrow \text{Sibling}(x, y)$$
- One's mother is one's female parent:

$$\forall x, y \text{ Mother}(x, y) \Leftrightarrow \text{Female}(x) \wedge \text{Parent}(x, y)$$

- 推理: 虽然比命题逻辑稍复杂, 但存在用于对一阶逻辑公式的知识库进行推理的程序。
- 自然语言: 一阶逻辑的表达能力强, 表明可以将自然语言和逻辑表达之间的转换自动化。这对各种应用非常有价值, 例如个人助理 (Siri)、问答系统以及计算机的通用交互。

SAT 的回溯 (Backtracking for SAT)

- 可满足性与推理的关系:

- $KB \models \alpha$ 当且仅当 $(KB \wedge \neg \alpha)$ 不可满足，即通过反证证明 α 。
- DPLL 算法用于检查命题逻辑句子的可满足性（SAT）。
- DPLL 算法类似于对约束满足问题（CSP）的回溯，但使用若干问题相关的信息/启发式方法，例如早期终止（Early Termination）、纯字母启发式（Pure symbol heuristic）和单子句启发式（Unit clause heuristic）。

DPLL

```

function DPLL-SATISFIABLE?(s) returns true or false
  inputs: s, a sentence in propositional logic

  clauses  $\leftarrow$  the set of clauses in the CNF representation of s
  symbols  $\leftarrow$  a list of the proposition symbols in s
  return DPLL(clauses, symbols, {})

```

```

function DPLL(clauses, symbols, model) returns true or false
  if every clause in clauses is true in model then return true
  if some clause in clauses is false in model then return false
  P, value  $\leftarrow$  FIND-PURE-SYMBOL(symbols, clauses, model)
  if P is non-null then return DPLL(clauses, symbols - P, model  $\cup$  {P=value})
  P, value  $\leftarrow$  FIND-UNIT-CLAUSE(clauses, model)
  if P is non-null then return DPLL(clauses, symbols - P, model  $\cup$  {P=value})
  P  $\leftarrow$  FIRST(symbols); rest  $\leftarrow$  REST(symbols)
  return DPLL(clauses, rest, model  $\cup$  {P=true}) or
    DPLL(clauses, rest, model  $\cup$  {P=false})

```